

ShintTools

Applied AI infrastructure for game pipelines

00 — INTRODUCTION

ShintTools for Unreal Engine 5

Native editor plugin for Unreal Engine 5.6 that runs the Code Validator, Asset Naming Bot and Quality Score modules entirely on the studio's own hardware. 100% offline. No telemetry. Source code, assets and scan results never leave the machine. ShintTools is under active development, so you may encounter some bugs or issues during use.

⚠ Free license limited to 500 assets per scan and the top 40 code-validator rules.

Contents

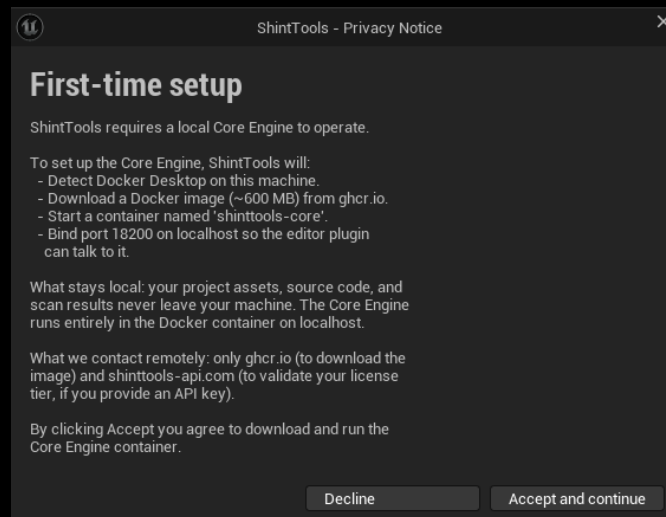
Installation.....	3
Opening the plugin.....	4
Overview.....	5
Code Validator.....	6

Panel actions.....	6
Issue row anatomy.....	6
Counters.....	7
C++ rules — Performance.....	7
C++ rules — Best Practices.....	7
C++ rules — Security.....	7
C++ rules — Maintainability.....	7
Blueprint rules.....	7
Asset Naming.....	9
Recommended prefixes.....	9
Settings.....	11
Support.....	12

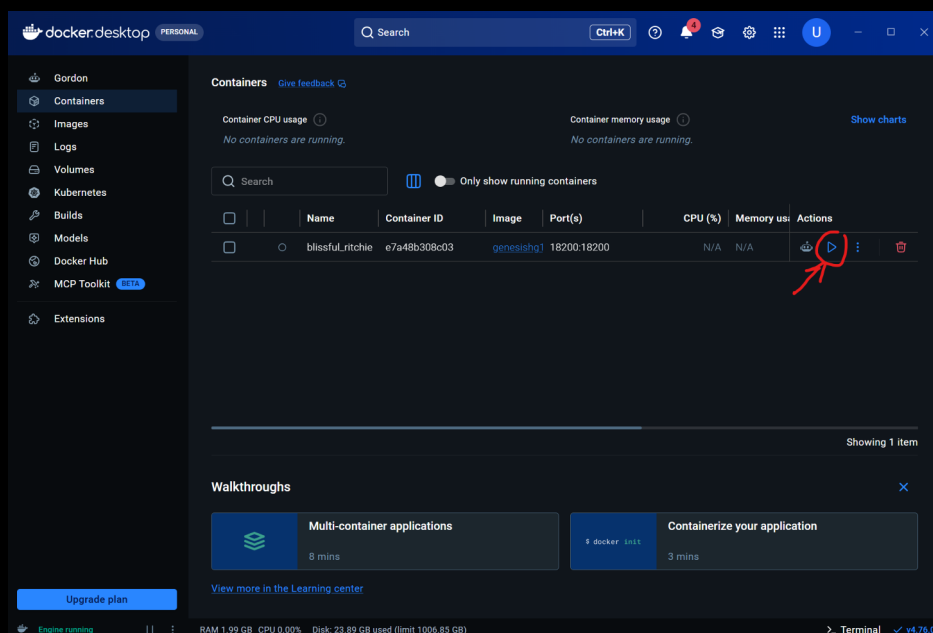
Installation

ShintTools for UE5 requires the following on the workstation:

- Unreal Engine 5.3 or newer (5.6 LTS recommended)
- Docker Desktop (runs the offline Core Engine)
- Python 3.11+ (only used by the installer; not needed at runtime)
- ~1.5 GB free disk for the Core Engine image



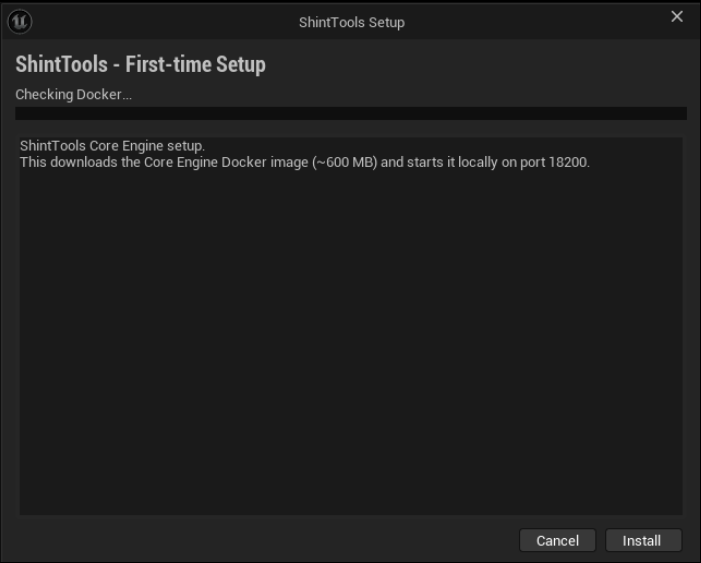
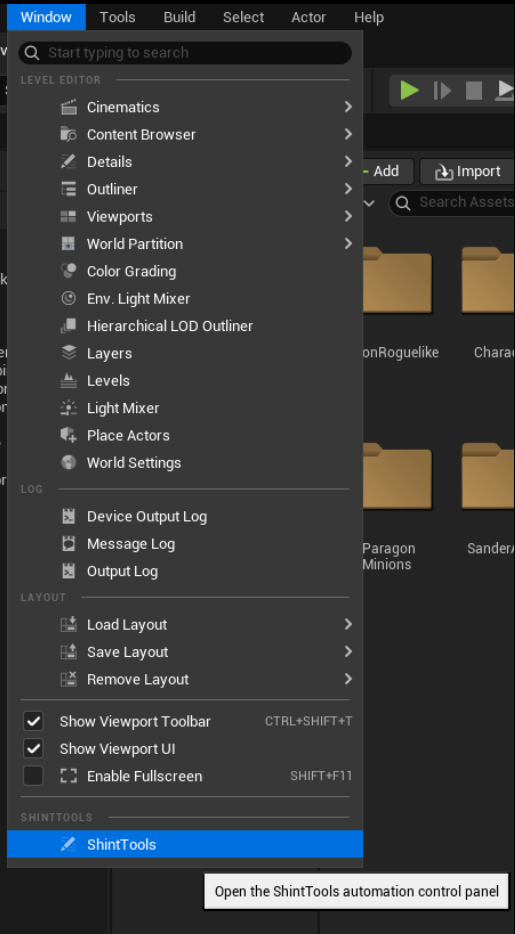
When the plugin loads for the first time the Core Engine wizard appears with a privacy-consent dialog. Click Accept to allow ShintTools to download the Docker image from ghcr.io and start the local AI container on port 18200. Subsequent launches only pull the delta. Docker Desktop must be running.



Opening the plugin

From the Unreal Editor menu bar choose Window > ShintTools, or press Ctrl+Shift+G. A dockable tab opens with a sidebar on the left listing every module:

SIDEBAR ITEM	WHAT IT SHOWS
Overview	Quality dashboard with 4 KPI tiles.
Code Validator	Static analysis for C++ and Blueprints.
Asset Naming	Naming-convention scan for /Game/ assets.
Settings	Paths, exclusions, license keys.
License button	Current tier (Free / Indie) — clicks open shint.tools.
Connection indicator	Green when the local Core Engine answers /health.



Overview

The Overview tab aggregates the most recent code + asset scan into four KPI tiles that the rest of the panel updates automatically.

TILE	READING
Quality Score	Overall project score (0–100). Colour-coded green / orange / red.
Total Issues	Code issues + asset naming violations combined.
Errors	Count of error-severity code findings (warnings excluded).
Files Scanned	Total files analysed across both modules.

Run any scan from the Code or Asset tabs to populate the tiles. The Quality Score includes a five-bucket breakdown — Performance, Security, Best Practices, Maintainability and Naming — that the Code Validator tab renders inline.

The screenshot shows the ShintTools application window. The title bar reads 'ShintTools' with standard window controls. The main window has a dark theme. At the top, a header bar shows 'Overview' on the left and 'Connected' status with a 'Free' button on the right. Below the header, a sidebar on the left contains 'ShintTools' and a list of tabs: 'Overview' (selected), 'Code', and 'Assets'. The main content area is titled 'PROJECT OVERVIEW' with a subtitle 'Quality Score and issue counters · last scan summary'. It features four KPI tiles: 'QUALITY SCORE' (displaying a horizontal bar), 'TOTAL ISSUES' (displaying '0'), 'ERRORS' (displaying '0' in red), and 'FILES SCANNED' (displaying '0'). Below these tiles is a large grey box with the text 'Run your first scan' and a subtext: 'Switch to the Code or Assets tab on the left to begin a scan. Scan results populate the tiles above.' At the bottom left of the window is a 'Settings' icon and label.

Code Validator

The Code Validator scans C++ source under your project's Source/ folder and every Blueprint asset under /Game/. Results land in a filterable list with severity badges, fix previews and a per-row LLM Explain entry point (Indie tier).

Panel actions

BUTTON	EFFECT
Scan Project	Recursive analysis of Source/ via the local Core Engine.
Scan Blueprints	Asset-Registry-driven scan of every /Game/ Blueprint.
Apply Fixes	Apply selected tree-sitter + BP fixes in one batch.
Explain	Open the per-issue LLM modal (Indie tier).
Send to Dashboard	Upload the scan result to shint.tools (Indie tier).
Export	Save the results list as JSON.
Select All	Tick every issue in the visible filter.
Deselect All	Untick every issue.
Filter dropdown	Filter by severity, category, file type or rule.

Issue row anatomy

ELEMENT	MEANING
Severity badge	Info / Warning / Error pill on the left.
Rule name	Human-readable label parsed from the rule docstring.
Message	What the rule found in this file.
File path	Click to reveal in the Content Browser or open in IDE.
Before / After	Diff preview for tree-sitter fixes (auto-fixable rules).
Fix button	Apply the suggested fix on disk + recompile.
Preview button	Show the diff without writing.
Explain button	Open the LLM explainer dialog (Indie tier only).

Counters

COUNTER	SOURCE
Files	Total files scanned in the last run.
Errors	Number of error-severity issues.
Warnings	Number of warning-severity issues.
Score	Project Quality Score (0-100).

C++ rules — Performance

- TickComponent overuse — actors with empty Tick + bCanEverTick=true
- GetWorld() in tight loops
- String concatenation in BeginPlay
- TArray::Add in render path
- Heavy logging in shipping builds

C++ rules — Best Practices

- Missing override keyword on virtual
- UFUNCTION/UPROPERTY missing required specifier
- Empty function body
- TODO / FIXME left in shipping code
- Unused #include

C++ rules — Security

- Hardcoded credentials in source
- Unchecked Cast<>
- Raw new without matching delete
- Buffer overflow risk (strcpy, sprintf)
- Insecure HTTP (http:// instead of https://)

C++ rules — Maintainability

- Long methods (>100 lines)
- High cyclomatic complexity
- Deep nesting (>4 levels)
- Long file (>800 lines)
- Public field on UObject without UPROPERTY

Blueprint rules

- BPP001 — empty Tick on bCanEverTick=true actor
- BPM001 — variable declared but never used
- BPM002 — orphan disconnected node

- BPB001 — Blueprint name does not follow convention
- BPB007 — public variable with no Category
- BPB003 — missing Super::BeginPlay call
- BPP004 — Cast<> chain longer than two hops

Auto-fixes for BPP001, BPM001, BPM002 and BPB007 apply inside the editor and recompile each affected Blueprint once at the end of the batch. ControlRigBlueprints are skipped automatically because their graph wiring is non-standard.

ShintTools

Overview

Code

Assets

CODE VALIDATOR

Analyse C++ source and Blueprints · review issues · apply fixes · send to dashboard

FILES
164

ERRORS
15

WARNINGS
203

QUALITY
97

Perf 100 · Sec 100 · BP 100 · Maint 100 · Naming 100

> Scan All C++ Source

> Scan All BP

RESULTS

All

All Categories

All Severities

Fixable Only

Select All

Deselect All

Warning

GetWorld without null-check

URogueAction :: RogueAction.cpp

GetWorld() called without null-check – guard with 'if (UWorld* W = GetWorld())'

✓ Apply

✗ Ignore

Preview

Explain

AUTO

Warning

GetWorld without null-check

URogueAction :: RogueAction.cpp

GetWorld() called without null-check – guard with 'if (UWorld* W = GetWorld())'

✓ Apply

✗ Ignore

Preview

Explain

AUTO

Warning

GetWorld without null-check

URogueAction :: RogueAction.cpp : 86

GetWorld() called without null-check – guard with 'if (UWorld* W = GetWorld())'

✓ Apply

✗ Ignore

Preview

Explain

AUTO

Warning

Magic number in expression

URogueActionComponent :: RogueActionComponent.cpp : 417

Magic number '2.0f' – extract to a named constexpr constant.

✓ Apply

✗ Ignore

Preview

Explain

AUTO

Error

Debug message left in code

URogueActionComponent :: RogueActionComponent.cpp : 417

GEngine debug message left in code – remove before shipping.

✓ Apply

✗ Ignore

Preview

Explain

AUTO

Info

TODO/FIXME comment

URogueActionComponent :: RogueActionComponent.cpp : 291

Preview

Explain

AUTO

Settings

Asset Naming

Asset Naming scans every asset under `/Game/` via the Asset Registry and flags names that do not follow the project's convention. Auto-rename uses `IAssetTools::RenameAssets` so all references (soft, hard and CoreRedirects) are kept in sync. Renames trigger a child-Blueprint recompile pass and an auto-save of the touched packages.

Recommended prefixes

ASSET TYPE	PREFIX
Blueprint	BP_
Widget Blueprint	WBP_
Texture2D	T_
Material	M_
Material Instance	MI_
Material Function	MF_
Static Mesh	SM_
Skeletal Mesh	SK_
Animation Sequence	AS_
Animation Blueprint	ABP_
Animation Montage	AM_
Blend Space	BS_
Sound Wave	SW_
Sound Cue	SC_
Niagara System	FX_
Particle System (legacy)	PS_
Level / Map	L_
Data Table	DT_
Data Asset	DA_
Curve	C_

Camera Shake	CS_
Physical Material	PM_

ShintTools

Asset Naming Bot

ConnectedFree

ShintTools

OverviewCodeAssets

ASSET NAMING BOT

Scan entire project · detect invalid names · apply UE5 rename (refs preserved) · send to dashboard

ASSETS

486

INVALID

500

TIME (s)

0.06

> Scan All Assets

Free tier: 500 of 1014 issues. Upgrade to Indie for full coverage.

Upgrade

RESULTS

All TypesSelect AllDeselect All

CurveFloat

/Game/ActionRoguelike/Audio/Blueprints/Curve_Music_CombatThreat_High

> Curve_Music_CombatThreat_High → CF_Curve_Music_CombatThreat_High

CurveFloat

/Game/ActionRoguelike/Audio/Blueprints/Curve_Music_CombatThreat_Low

> Curve_Music_CombatThreat_Low → CF_Curve_Music_CombatThreat_Low

CurveFloat

/Game/ActionRoguelike/Audio/Blueprints/Curve_Music_CombatThreat_Medium

> Curve_Music_CombatThreat_Medium → CF_Curve_Music_CombatThreat_Medium

CurveFloat

/Game/ActionRoguelike/Audio/Blueprints/Curve_Music_ProximityThreat_High

> Curve_Music_ProximityThreat_High → CF_Curve_Music_ProximityThreat_High

CurveFloat

/Game/ActionRoguelike/Audio/Blueprints/Curve_Music_ProximityThreat_Low

> Curve_Music_ProximityThreat_Low → CF_Curve_Music_ProximityThreat_Low

CurveFloat

/Game/ActionRoguelike/Audio/Blueprints/Curve_Music_ProximityThreat_Medium

> Curve_Music_ProximityThreat_Medium → CF_Curve_Music_ProximityThreat_Medium

MapBuildDataRegistry

/Game/ActionRoguelike/Maps/TestLevel_BuiltData

> TestLevel_BuiltData → TestLevel_BuiltData

StaticMesh

/Game/ActionRoguelike/Maps/_GENERATED/tomlo/CubeGridToolOutput_84F011D5

> CubeGridToolOutput_84F011D5 → SM_CubeGridToolOutput_84F011D5

StaticMesh

/Game/ActionRoguelike/Maps/_GENERATED/tomlo/CubeGridToolOutput_9DD264EF

>

Settings

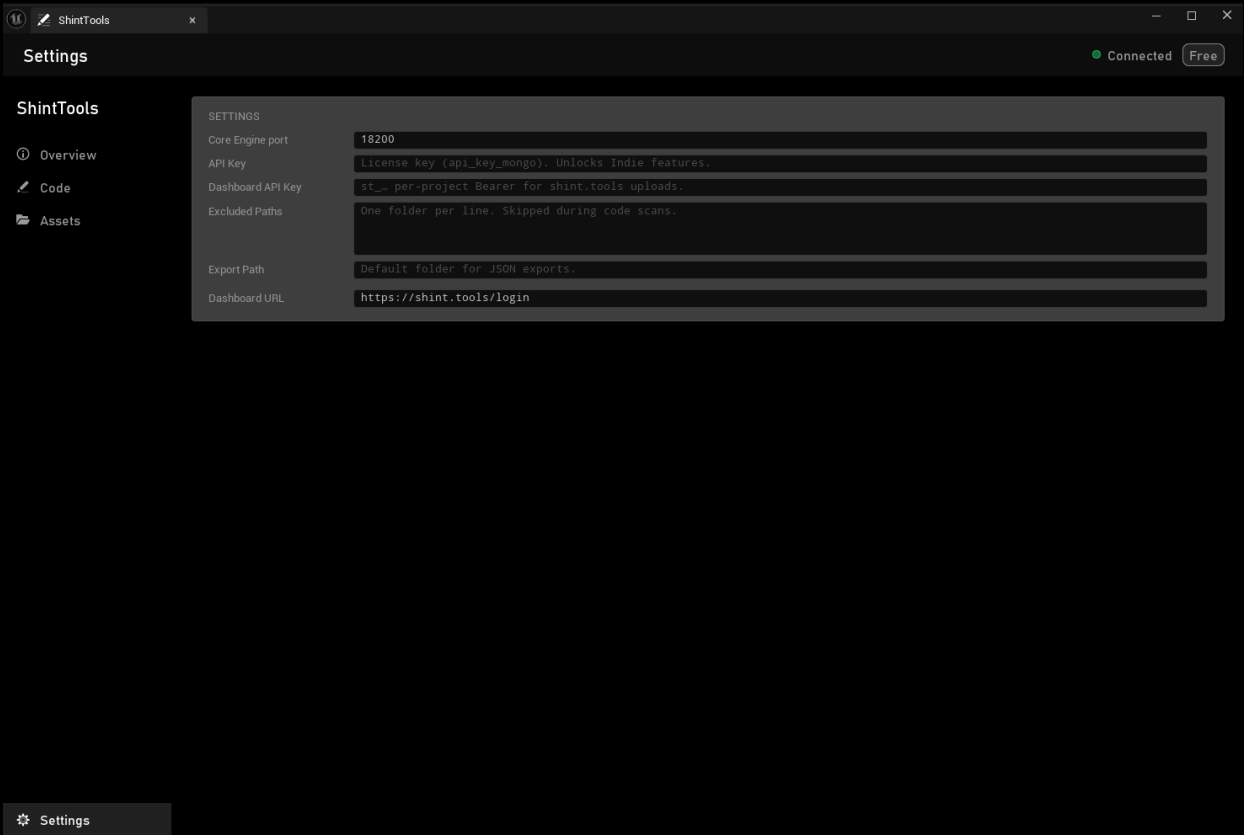
Settings

The Settings tab persists into shinttools.config.json at the project root.

FIELD	PURPOSE
Core Engine port	Local port the AI listens on. Default 18200.
API Key	License key for Indie features (api_key_mongo).
Dashboard API Key	Per-project Bearer for shint.tools uploads.
Excluded Paths	Folders the validators skip during scans.
Export Path	Default folder for JSON exports.

WHERE THE CONFIG LIVES

Path: <YourProject>/shinttools.config.json. Safe to commit to version control — no secrets are written there by default; the API keys are local user settings entered through the panel.

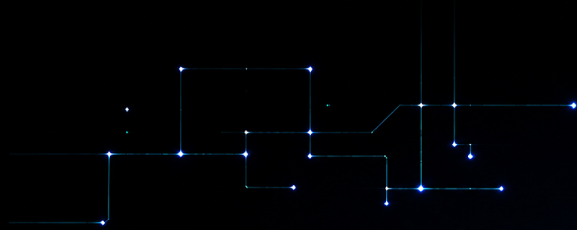


Support

Bug reports, feature requests and licence administration:

- Email: support@shint.tools
- Dashboard: <https://shint.tools/login>

If you find ShintTools useful, please leave a review on the Fab marketplace — feedback drives the roadmap.



ShintTools

Applied AI infrastructure for game pipelines